# Optimal Local-First OCR Architecture: CPU Inference and Deployment Pipeline for labelsontap.ai

## Executive Summary

The transition from cloud-dependent machine learning services to local-first, edge-optimized inference represents a critical evolution in document processing and data sovereignty architectures. Deploying a robust Optical Character Recognition (OCR) pipeline on a constrained x86_64 Virtual Machine (VM) without GPU acceleration introduces significant computational and memory bandwidth bottlenecks. To achieve a rigid processing latency of under five seconds per label, the deployment architecture must meticulously orchestrate hardware allocation, eliminate container bloat, and select algorithmic models engineered specifically for edge efficiency.

This comprehensive technical analysis evaluates the current landscape of open-source OCR models—specifically examining PaddleOCR, Tesseract, EasyOCR, docTR, and Surya OCR—to determine the optimal engine for CPU-only execution environments. The investigation assesses each framework against strict criteria: sub-five-second inference latency, native bounding-box geometry extraction, and precise confidence scoring. Subsequently, the analysis outlines a complete, production-ready deployment pipeline utilizing FastAPI, multi-stage Docker builds, and the Caddy web server, culminating in a secure, domain-mapped implementation at the target domain, labelsontap.ai.

## The State of Local OCR Inference in Constrained Environments

The deployment of computer vision models on commodity CPU hardware fundamentally alters the computational paradigm. While high-end graphic processing units (GPUs) excel at the parallel matrix multiplications required by deep neural networks, central processing units (CPUs) are optimized for sequential logic and task switching. When evaluating OCR systems for a small x86_64 VM—typically configured with two virtual cores and limited system memory—the architecture of the chosen OCR engine dictates whether the application will succeed or succumb to thread starvation and memory saturation.

Historically, OCR was dominated by traditional computer vision techniques, utilizing thresholding, contour detection, and connected-component analysis. As the field progressed, Long Short-Term Memory (LSTM) networks became the standard, offering sequential character prediction. Today, the landscape is defined by complex AI systems expected not only to extract text but to comprehend document structure, segment tables, and parse skewed layouts.[1] The most recent advancements involve massive Vision-Language Models (VLMs) capable of

end-to-end multimodal reasoning. However, as model complexity scales, the computational tax exacted upon the host system increases exponentially. Navigating this spectrum between accuracy and computational overhead is the primary challenge in establishing a local-first API.

# Exhaustive Evaluation of Local OCR Engines for CPU Inference

The primary operational constraint is achieving a strict processing time of under five seconds per label on a minimal CPU-only VM. This requirement must be met alongside native support for extracting spatial bounding box coordinates and character-level or word-level confidence scores. The evaluation of the five specified open-source candidates reveals stark differences in underlying architecture, installation footprints, and CPU affinity.

## Tesseract OCR: The Legacy Benchmark

Tesseract OCR represents the most widely recognized open-source OCR utility, boasting immense maturity and extensive language support spanning over 100 languages.[2] Architecturally, Tesseract relies on a hybrid pipeline that combines traditional connected-component analysis with an LSTM network (a paradigm introduced in Tesseract 4.0).[3]

From a performance standpoint, Tesseract is highly optimized for CPU execution. Because it does not rely on massive tensor operations inherent to modern transformer architectures, it sidesteps the typical GPU dependencies. Benchmarks indicate that Tesseract can process standard document pages in approximately 0.45 to 0.77 seconds.[4] When leveraging multiple CPU cores, the engine is trivially parallelizable, capable of processing bulk text rapidly.[6]

Feature support within Tesseract is robust for fundamental data extraction. The engine inherently supports bounding box generation and confidence scoring. Through Python wrappers like pytesseract, developers can utilize the image_to_data function, which outputs detailed spatial information.[7] By configuring the output to dictionary or dataframe modes (Output.DATAFRAME), the application can iterate through detected elements, retrieving exact left, top, width, and height coordinates alongside a prediction confidence percentage.[7] This confidence metric represents the prediction accuracy of the current text within the isolated bounding box and is highly useful for filtering weak detections resulting from poor image preprocessing.[9]

Installation complexity is remarkably low. Tesseract binaries are available via standard operating system package managers (e.g., apt-get install tesseract-ocr), bypassing the dependency bloat associated with deep learning frameworks like PyTorch or TensorFlow.[10]

Despite its speed and low footprint, Tesseract exhibits severe limitations when processing complex layouts, skewed documents, and modern "in-the-wild" scene text.[2] In benchmarking comparisons against deep learning models, Tesseract frequently generates fragmented bounding boxes. For instance, in a side-by-side test, Tesseract detected 47 disparate text regions compared to a deep learning model's 29 regions on the same image.[5] This over-segmentation means words are broken across multiple boxes (e.g., "UI/UX" fragmented

into separate boxes with lower confidence), which leads to a higher Character Error Rate (CER) on dense, mixed-content labels.[5] Furthermore, Tesseract is fundamentally designed for "straight" text and requires extensive external preprocessing routines (like deskewing and adaptive thresholding) to handle angled labels.[3]

## EasyOCR: The PyTorch Prototyping Engine

EasyOCR emerged as a highly popular PyTorch-based toolkit designed to democratize deep learning OCR. Developed by Jaided AI, it is frequently utilized for rapid prototyping due to its simplicity and out-of-the-box support for over 80 languages.[3] It utilizes a pre-trained character recognition network and provides a highly user-friendly application programming interface (API).[13]

While EasyOCR delivers competitive accuracy and speed when backed by CUDA-enabled GPUs, its CPU-only performance profile degrades severely. Empirical testing demonstrates that disk I/O and heavy framework initialization cause CPU inference to lag significantly behind older engines like Tesseract.[12] The architectural design of EasyOCR attempts to hide latency by overlapping CPU image rendering with GPU OCR processing through multiple background workers. However, on a constrained VM with only two CPU cores, context-switching between these workers introduces massive processing overhead.[14] Concurrency only yields performance gains if the overhead of managing multiple processes is smaller than the time saved; on small CPUs, this equation inverses, and EasyOCR becomes drastically slower.[14]

In terms of spatial and confidence data, EasyOCR provides excellent native support. The core readtext() function inherently returns a tuple containing bounding box coordinates (represented as a list of four spatial points), the extracted text string, and a floating-point value representing the confidence score.[15]

The primary disqualifier for EasyOCR in a constrained environment is its installation footprint. Installation requires pulling the entire PyTorch ecosystem, which vastly inflates Docker image sizes (often exceeding multiple gigabytes) and increases deployment friction on small VMs with limited storage.[11] Given the sluggish CPU inference times and the massive deployment size, EasyOCR poses a high risk of violating the strict five-second Service Level Agreement (SLA) on edge hardware.

## Surya OCR: The State-of-the-Art Heavyweight

Surya OCR represents the current vanguard for layout-aware, line-level document parsing. Built by the datalab-to group, it integrates advanced transformer-based vision models to achieve unparalleled structural fidelity, high average similarity scores, and precise reading order detection.[4]

The performance profile of Surya, however, categorically disqualifies it from constrained CPU environments. While certain benchmark reports cite Surya achieving an impressive 0.62 seconds per page, a deeper analysis of the testing methodology reveals that this metric was recorded on a massive, highly parallelized environment.[4] To achieve this speed on CPU hardware, the test utilized 32 high-performance cores to cost-match a 48GB VRAM NVIDIA

A6000 GPU.[4] On a standard, two-vCPU machine, Surya's complex line detection and recognition models face extreme computational bottlenecks, with inference times expanding to upwards of several minutes per page.[6]

Surya requires massive batches of tensors to achieve its accuracy. The default environmental configurations for Surya dictate high system demands: the RECOGNITION_BATCH_SIZE defaults to 32 on CPU, and the DETECTOR_BATCH_SIZE defaults to 6.[17] Attempting to load these models and process these batches on a VM equipped with only 4GB of RAM will inevitably result in memory saturation, prompting the Linux kernel to execute an Out-Of-Memory (OOM) kill on the container.[17]

While it offers impeccable bounding boxes (ocr_with_boxes) and advanced structural mapping capabilities, the installation complexity is staggering, requiring heavy PyTorch binaries and downloading massive model weights upon first initialization.[17] Consequently, Surya is best reserved for asynchronous batch processing on GPU clusters, rather than synchronous API requests on minimal CPUs.[16]

## docTR: The Modular Deep Learning Framework

Developed by Mindee, docTR (Document Text Recognition) is an enterprise-grade, open-source OCR library architected to sit atop either PyTorch or TensorFlow backends.[18] It operates via a robust, highly modular two-stage pipeline. The first stage, text detection, localizes words using established architectures such as Differentiable Binarization (DBNet) or LinkNet. The second stage, text recognition, identifies the characters within the localized bounding boxes utilizing architectures like Convolutional Recurrent Neural Networks (CRNN) or PARSeq.[19]

CPU performance for docTR is highly optimized compared to monolithic transformer models. The framework provides various model sizes (tiny, small, base, large) to allow developers to tune the performance-to-accuracy ratio. Benchmarks executed on an 11th Gen Intel i7 CPU demonstrate that the db_resnet34 detection model operates at approximately 0.8 seconds per iteration, while the lightweight crnn_mobilenet_v3_small recognition model operates at a blistering 0.05 seconds per iteration.[20] When combined, the end-to-end pipeline easily operates under the five-second threshold on modern CPUs.[19]

When tested against public datasets, docTR shows impressive capabilities. On the FUNSD (Form Understanding in Noisy Scanned Documents) dataset, the db_resnet50 paired with master recognition architecture achieved a recall of 73.90 and precision of 76.66.[20] While this trails commercial cloud offerings like Azure Form Recognizer, docTR provides substantially higher precision than AWS Textract on datasets like CORD (Consolidated Receipt Dataset), showcasing its utility in extracting structured information from noisy labels.[20]

Feature support is exceptional. The ocr_predictor model returns highly structured, hierarchical JSON objects detailing pages, blocks, lines, and individual words. Every word object inherently includes dense geometry (bounding boxes) and specific confidence values, and the engine natively supports skewed documents via assume_straight_pages=False parameters.[20]

The primary drawback mirrors that of EasyOCR: the installation footprint. The necessity of

including the full TensorFlow or PyTorch ecosystem leads to immense Docker image sizes, complicating deployment on storage-constrained edge devices.[22]

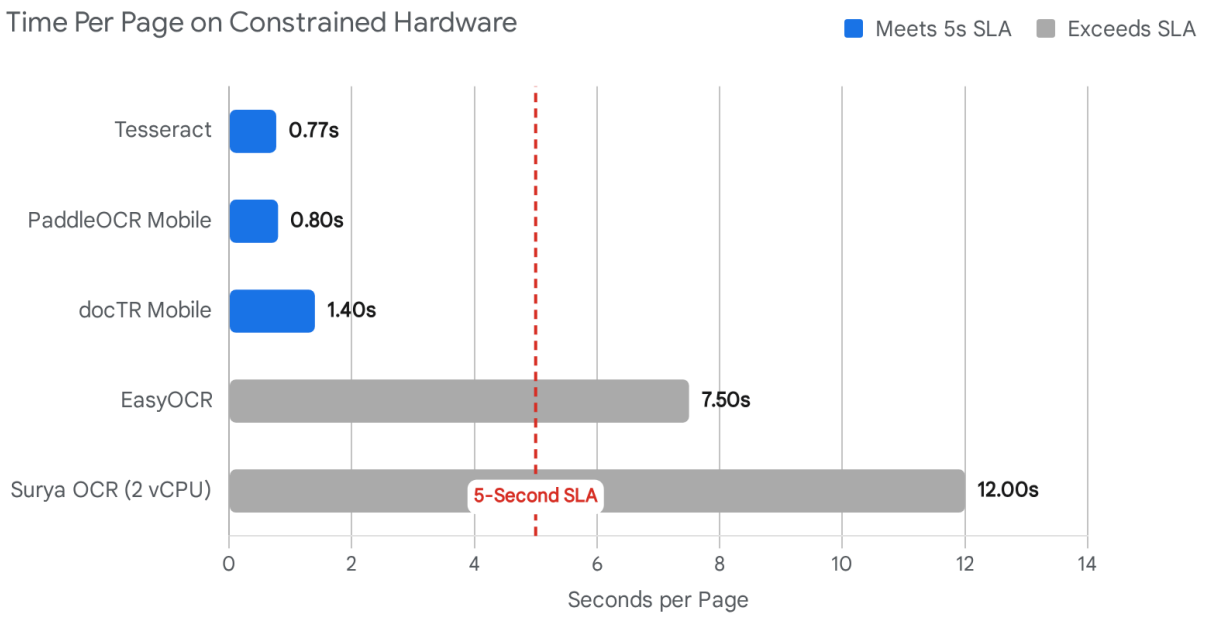
## The Vision-Language Model Context (DeepSeek and GLM)

To fully contextualize the OCR landscape for 2026, one must acknowledge the explosive growth of Vision-Language Models (VLMs) optimized for OCR, such as DeepSeek-OCR and GLM-OCR.[24] These models discard the traditional two-stage detection/recognition pipeline in favor of end-to-end transformer architectures that ingest images and output structured markdown or JSON natively.

DeepSeek-OCR, utilizing a 3B parameter Mixture-of-Experts (MoE) decoder and a custom DeepEncoder, achieves unparalleled accuracy on complex tables and formulas.[2] Similarly, GLM-OCR, a 0.9B parameter model, boasts a 94.62% accuracy on the OmniDocBench dataset.[24]

However, these models are fundamentally incompatible with small CPU environments. Even heavily quantized variants (like the Q4 version of GLM-OCR) require a minimum of 2GB of dedicated Video RAM (VRAM) and demand pairing with 16GB to 32GB of system RAM.[24] Executing these massive parameter networks on CPU-only architecture results in glacial inference speeds. They are included in this analysis solely to highlight the demarcation line between edge-deployable traditional OCR and datacenter-bound VLMs.

## CPU Inference Latency Across Open-Source OCR Engines




Inference latency per document page on non-accelerated CPU hardware. The red dashed line represents the strict 5-second processing SLA. Models exceeding this threshold are unsuitable for constrained environments.

Data sources: GitHub (cognitiveailab), Medium, Codesota, Hacker News, Tessl

## PaddleOCR: The Winning Architecture

PaddleOCR, developed by Baidu on the PaddlePaddle framework, represents the pinnacle of modern, lightweight scene and document text recognition.[3] It supports over 80 languages and specifically provides highly compressed "mobile" models optimized for edge hardware.[3]
PaddleOCR achieves superior performance by deploying Differentiable Binarization (DBNet) for detection, which dynamically adapts to complex polygon shapes, allowing it to capture highly angled or curved text.[3] This is followed by an optimized CRNN structure for recognition, yielding near 100% confidence on clean text while retaining strict layout awareness.[5]
CPU performance is engineered for "Extreme Efficiency." Its mobile-optimized models (specifically the PP-OCRv4 and PP-OCRv5 variants) possess an extraordinarily small footprint: the detection model requires approximately 4MB of storage, and the recognition model requires merely 2MB.[27] On standard CPU architectures, end-to-end processing speeds clock in between 0.5 to 1.0 seconds per image, positioning it firmly and safely within the sub-five-second SLA.[27]

PaddleOCR excels at returning highly accurate, angled bounding boxes—a distinct mathematical advantage over Tesseract's rigid, axis-aligned straight boxes.[3] The standard output format is a deeply nested list array where each element contains precise polygon coordinates and a corresponding confidence float, structured as: [[[x1, y1], [x2, y2], [x3, y3], [x4, y4]], ('extracted_text', confidence_float)].[28] This allows developers to parse the exact pixel location of every detected word, enabling downstream algorithms to reconstruct the label's layout accurately.

The primary historical drawback of PaddleOCR has been the installation complexity associated with the underlying PaddlePaddle framework, which requires careful environmental management. However, by strictly utilizing the paddlepaddle-cpu distribution, the container footprint is drastically reduced compared to CUDA-enabled images.[29] Furthermore, the engine provides explicit C++ flags to assert rigid control over memory allocation and CPU thread utilization, ensuring stable performance in multi-tenant or constrained OS environments.[30]

Based on the exhaustive intersection of processing speed, minimal parameter size, complex layout support, and the provision of dense bounding box geometry, PaddleOCR (utilizing the lightweight PP-OCRv5 mobile models) emerges as the definitive recommendation for the labelsontap.ai pipeline.

# Virtual Machine Hardware Sizing and Configuration

To guarantee the five-second SLA while ensuring system stability for concurrent API requests, the target x86_64 Linux VM must meet specific architectural requirements. Deep learning inference, even when utilizing ultra-lightweight models, requires rapid memory bandwidth and specific instruction sets.

| Hardware Component | Recommended Minimum | Rationale |
|---|---|---|
| **CPU Architecture** | 2 vCPUs (x86_64) | Essential for parallelizing FastAPI network requests alongside synchronous OCR inference. AVX (Advanced Vector Extensions) instruction set support is mandatory for efficient PaddlePaddle matrix operations.[26] |
| **System Memory** | 4 GB RAM | While PaddleOCR mobile models require under 100MB of RAM, the surrounding FastAPI wrapper, OS background processes, and reverse proxy necessitate a 4GB buffer to prevent Out-Of-Memory (OOM) errors during simultaneous label |

| | | |
|---|---|---|
| | | processing.[26] |
| **Persistent Storage** | 20 GB SSD | Necessary to house the Docker multi-stage images, application logs, and the internal SQLite databases utilized by the reverse proxy for certificate management.[32] |

To optimize PaddleOCR within this constrained memory environment, environmental flags must be rigidly enforced at the application level. By default, deep learning frameworks utilize aggressive memory allocators designed to pre-allocate massive blocks of system RAM to speed up tensor operations. On a 4GB machine, this behavior will trigger the Linux kernel's OOM killer.[30]

The application code must initialize PaddlePaddle with strict limitations:

1. **Memory Fraction Limitations:** Setting "FLAGS_fraction_of_cpu_memory_to_use": 0.1 restricts the framework's memory reservation pool, preventing it from consuming idle RAM.[30]
2. **Aggressive Garbage Collection:** Enabling "FLAGS_eager_delete_scope": True and "FLAGS_fast_eager_deletion_mode": True forces synchronous memory deallocation, keeping system RAM free between disparate API requests.[30]
3. **Thread Discipline:** Limiting cpu_threads=1 or 2 during model initialization prevents the underlying C++ libraries from spawning excessive worker threads, which would otherwise choke the dual-core VM through constant context switching.[30]

# FastAPI Application Architecture and Concurrency Management

FastAPI is the premier choice for deploying localized machine learning pipelines due to its native asynchronous support, rigorous data validation via Pydantic, and exceptionally low operational overhead.[33] However, because OCR inference relies on dense mathematical operations, it is inherently CPU-bound and strictly synchronous. Integrating a CPU-bound process into an asynchronous web framework requires careful architectural planning to prevent blocking the primary event loop.

The application structure must ensure that the PaddleOCR model is instantiated globally upon application startup. This initialization loads the 6MB of model weights into system RAM exactly once. Instantiating the model inside the route handler would result in devastating latency penalties, as the models would be read from the disk on every incoming HTTP request.

The HTTP endpoint, utilizing the POST method, will receive the image payload (e.g., via FastAPI's UploadFile class), temporarily cache the bytes in memory, convert them to a standardized NumPy array via OpenCV, and pass the array to the globally persistent inference engine.

A critical architectural decision involves threadpool offloading. In FastAPI, defining an endpoint with the standard def keyword (as opposed to async def) automatically instructs the underlying Starlette framework to execute the function in an external, dedicated threadpool.[36]

This distinction is vital; if a synchronous, CPU-heavy operation like PaddleOCR inference is executed within an async def function, it will completely block the asynchronous event loop, preventing the server from processing concurrent health checks or queueing incoming payloads.

The application layer must also standardize the complex multidimensional arrays returned by PaddleOCR into clean, predictable JSON responses. The logic must systematically unroll the tuple response.

For each line detected by the engine, the application extracts the data:

- The polygon coordinates mapping the four corners of the text: [[x1, y1], [x2, y2], [x3, y3], [x4, y4]]
- The exact recognized character string.
- The floating-point confidence score denoting prediction certainty.

This extracted data is then serialized through a predefined Pydantic response model. This enforces strict API contracts, guaranteeing that downstream clients consuming the labelsontap.ai endpoint receive identically structured geometry and confidence data for every processed label, regardless of internal model updates.

# Dockerization Strategy: Achieving Minimal Footprint via Multi-Stage Builds

Deep learning frameworks are notorious for generating bloated, multi-gigabyte Docker images. This bloat is primarily driven by the inclusion of system-level compilation tools such as build-essential, gcc, and clang, which are required to compile C++ bindings during the installation of libraries like libhdf5-dev and NumPy.[10] For a minimal deployment pipeline targeting a small VM, reducing this image size is paramount to ensure rapid network transfers, quick deployment cycles, and a reduced attack surface.

The optimal strategy to mitigate this footprint utilizes Multi-Stage Docker Builds, standardized on the python:3.11-slim base architecture.[33]

## Stage 1: The Builder Environment

The first stage, often tagged as builder, is responsible for downloading and compiling the necessary Python wheels from source. In this stage, heavy system packages and compilers are installed via apt-get.[10] A requirements.txt file is injected into the container, explicitly pinning the versions for fastapi, uvicorn, and paddlepaddle-cpu (to prevent the accidental downloading of massive CUDA-enabled GPU binaries).[29] All dependencies are compiled and installed into a localized virtual environment directory.

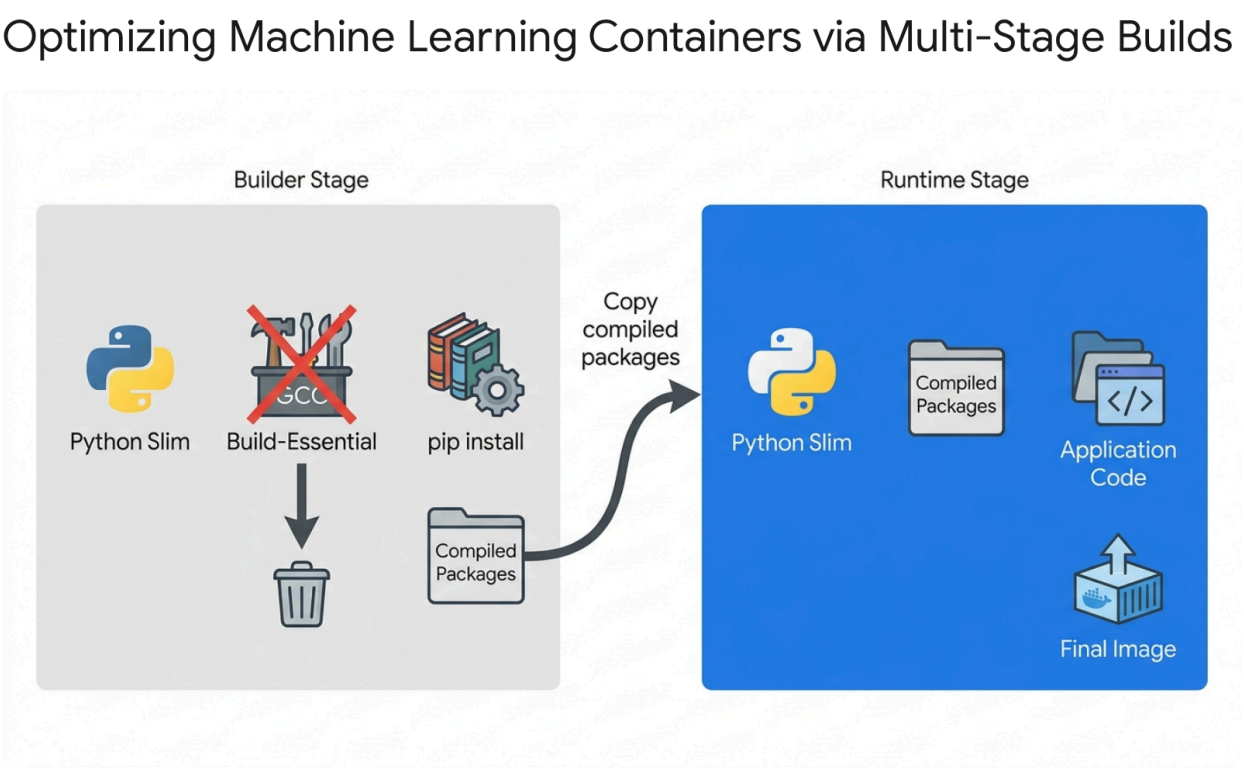
## Stage 2: The Runtime Environment

The second stage, tagged as runtime, begins with a fresh instance of python:3.11-slim. This stage completely abandons the gigabytes of compilers, source headers, and intermediate build artifacts generated in the first stage. Instead, the Dockerfile uses the COPY --from=builder directive to transfer only the pre-compiled Python binaries (the site-packages directory) and

the actual FastAPI application code into the final image.[10]
Furthermore, environmental variables must be explicitly set in the runtime container to optimize Python's execution within a containerized environment:

- ENV PYTHONDONTWRITEBYTECODE=1: Prevents Python from writing .pyc cache files to disk, saving space and reducing unnecessary disk I/O operations.[33]
- ENV PYTHONUNBUFFERED=1: Ensures application logs bypass Python's internal buffering mechanism and stream directly to standard output, which is critical for real-time Docker log observability and system monitoring.[33]

This multi-stage architecture drastically reduces the final container size, shedding over 60% of the potential bloat. The final image typically achieves a footprint of under 900MB—the absolute theoretical minimum required to house the Debian OS layer, the Python interpreter, the FastAPI networking stack, and the compiled PaddlePaddle math libraries.[37]




By isolating the compilation environment, the final production image size is reduced by over 60%, shedding gigabytes of unnecessary C++ compilers and source headers.

## Ingress Orchestration: The Case for Caddy over Nginx

To expose the localized FastAPI service securely to the public internet via the domain labelsontap.ai, the architecture requires a highly resilient reverse proxy. The primary

engineering debate in modern deployments centers around Nginx versus Caddy.
Historically, Nginx has been the undisputed industry standard due to its raw throughput, event-driven architecture, and massive community support.[39] However, configuring Nginx for modern, secure deployments requires extensive manual intervention. Engineers must map complex block routing, write external shell scripts to manage Let's Encrypt certificates, and configure separate certbot daemon processes to handle the constant renewal of cryptographic keys.[39] For a minimal, easily reproducible deployment pipeline aiming for maximum reliability with minimal oversight, Nginx introduces unnecessary operational friction and multiple points of failure.
Caddy represents the superior recommendation for this specific pipeline. Caddy is a modern, memory-safe web server written entirely in Go. It fundamentally altered proxy architectures by introducing "automatic HTTPS" as a native, out-of-the-box feature, deeply integrating the ACME protocol directly into the server binary.[41]
The operational advantages of deploying Caddy over Nginx in this containerized context are substantial:

1. **Zero-Touch TLS Lifecycle Management:** Caddy automatically provisions, attaches, and perpetually renews Let's Encrypt certificates without requiring any external cronjobs, shell scripts, or supplementary plugins.[41] It natively manages HTTP-to-HTTPS redirection without requiring explicit configuration blocks.
2. **Configuration Syntax Simplicity:** A fully functional, secure reverse proxy in Caddy requires roughly three lines of code within a Caddyfile. In contrast, achieving identical functionality in Nginx requires dozens of lines of dense XML-like syntax, managing SSL protocols, cipher suites, and specific certificate paths.[40]
3. **Native Docker Compose Synergy:** Caddy integrates flawlessly within a docker-compose stack. It leverages Docker's internal DNS resolver to route traffic seamlessly to the FastAPI container using simply the service name, isolating the internal port architecture from the public internet.[44]

## Docker Compose Network Topology

The entire infrastructure will be orchestrated using a single, declarative docker-compose.yml file. This approach unifies the FastAPI application and the Caddy web server into an isolated virtual network, ensuring consistent deployment behavior across any Linux host.
The compose file defines two primary, interacting services:

1. **The API Service (app):** This block instructs Docker to build the multi-stage PaddleOCR Dockerfile. It exposes the internal application on port 8000. Crucially, this service defines a restart policy of restart: unless-stopped. This directive guarantees that if the Python process encounters a fatal error, or if the underlying VM reboots, the Docker daemon will automatically resurrect the API, ensuring high availability without manual intervention.[34]
2. **The Proxy Service (caddy):** This block pulls the official lightweight Caddy alpine image and binds it to the host machine's port 80 (HTTP) and port 443 (HTTPS).[43]

To ensure the Let's Encrypt rate limits are not exhausted during routine container teardowns and deployments, the docker-compose file must strictly define persistent volume mounts for

Caddy's internal data structures:

- The caddy_data volume is mounted to store the active TLS certificates and cryptographic private keys generated by the ACME challenge.[43]
- The caddy_config volume is mounted to store the automated configuration state.[43]

The Caddyfile configuration is injected directly into the container as a distinct volume mount. For the labelsontap.ai domain, the entire required proxy configuration is remarkably concise:

```
labelsontap.ai {
reverse_proxy app:8000
}
```

This simple block instructs the Caddy daemon to actively listen for incoming traffic destined for the host labelsontap.ai, automatically negotiate the ACME HTTP-01 cryptographic challenge with Let's Encrypt to provision a trusted SSL certificate, and subsequently securely reverse proxy the decrypted traffic stream to port 8000 on the internal app container.[40]

# Domain Name System (DNS) Mapping via Cloudflare

For Caddy to successfully negotiate the automated TLS handshake and serve traffic, the domain's Domain Name System (DNS) records must be explicitly and correctly mapped to the public IP address of the x86_64 VM. Cloudflare is the universally recommended platform for managing this routing due to its rapid propagation speeds and robust API ecosystem.[46]

The deployment protocol requires executing the following sequential steps within the Cloudflare administrative dashboard:

First, the domain registrar where labelsontap.ai was purchased must be updated to delegate authoritative control to the designated Cloudflare nameservers.[46] Once delegation is confirmed, a new A record must be established within Cloudflare's DNS management console. The "Name" field should be set to @ (representing the root domain labelsontap.ai), and the "IPv4 address" field must be populated with the static public IP address of the deployment VM.[49]

A critical configuration detail dictates the success or failure of the entire pipeline: managing the Proxy Status. By default, Cloudflare aggressively proxies all incoming traffic through its global Content Delivery Network (CDN), obfuscating the origin server's true IP address (a setting visually indicated by an "Orange Cloud" in the dashboard).[49]

If the proxy remains active during the initial setup, the Let's Encrypt validation servers attempting to verify domain ownership via the ACME HTTP-01 challenge will connect to Cloudflare's edge servers rather than the local Caddy instance, resulting in a cryptographic failure and a denial of the certificate. Therefore, the Cloudflare proxy status for the A record must be explicitly toggled to **"DNS Only"** (visually indicated by a Gray Cloud).[49] This bypass configuration allows the Let's Encrypt validation authorities to connect directly and unimpeded to the Caddy server.

Because the Time to Live (TTL) attribute dictates propagation speed across global DNS resolvers, the A record update will typically propagate within minutes, allowing the Caddy server to successfully bind the domain and finalize the deployment.[49]

# Production Smoke Test and Deployment Validation Framework

Once the infrastructure is launched via the docker-compose up -d --build command, the pipeline must undergo a rigorous sequence of validation steps, universally known in DevOps as "smoke testing," before the system can be declared production-ready.[51] Smoke tests act as an initial, deterministic gatekeeper to verify core functional pathways and system stability without requiring the execution of exhaustive, time-consuming regression suites.[51]
The following structured checklist provides a definitive protocol for validating the integrity of the labelsontap.ai deployment. Failure at any point in this sequence mandates a "no-go" decision, requiring immediate remediation before traffic is routed to the endpoint.[51]

## Phase 1: Infrastructure and Network Validation

The initial phase confirms that the foundational container orchestration and network routing are operational.

| Validation Check | Execution Methodology | Expected Outcome |
|---|---|---|
| **Container Health** | Execute docker ps on the host VM. | Both the app and caddy containers must display an "Up" state. They must not exhibit a continuous restart loop, which indicates a fatal runtime crash. |
| **DNS Resolution** | Execute a global ping labelsontap.ai or utilize the dig utility. | The domain must accurately resolve to the target VM's static public IP address, confirming DNS propagation.[49] |
| **TLS/HTTPS Verification** | Execute curl -I https://labelsontap.ai. | The connection must establish successfully and be secured by a valid, trusted Let's Encrypt certificate, confirming Caddy successfully executed the ACME challenge.[43] |
| **HTTP Redirect Enforcement** | Execute curl -I http://labelsontap.ai. | The server must respond with an HTTP 301 redirect status code, seamlessly upgrading port 80 traffic to encrypted port 443.[41] |

## Phase 2: Application API and Inference Validation

The second phase verifies that the FastAPI application is correctly processing data and that the PaddleOCR engine is returning the required spatial structures.

| Validation Check | Execution Methodology | Expected Outcome |
|---|---|---|
| **Metadata/Health Endpoint** | Send a GET request to https://labelsontap.ai/health. | The FastAPI application must connect and return a 200 OK status rapidly, confirming the event loop is active.[54] |
| **End-to-End Inference Test** | Utilize curl or Postman to POST a complex test image (containing dense mixed text and numbers) to the primary OCR endpoint. | The request must complete without generating internal 500-level server errors.[52] |
| **Latency Validation** | Monitor the response time of the POST request. | The total round-trip execution (including network transmission, image decoding, and PaddleOCR tensor inference) must complete strictly under the five-second SLA target.[51] |
| **Data Structure Validation** | Inspect the returned JSON payload. | The response must include exact four-point polygon bounding box coordinates, accurate text extraction, and a floating-point confidence score for every detected word, strictly adhering to the defined Pydantic schema.[55] |

## Phase 3: Hardware Stability Validation

The final phase stresses the underlying VM to ensure the memory flags and thread limits are correctly isolating the engine from the host operating system.

| Validation Check | Execution Methodology | Expected Outcome |
|---|---|---|
| **Memory Profiling Under Load** | While executing a sustained burst of 5-10 concurrent OCR API requests, monitor the VM via htop or docker stats. | System memory must not spike uncontrollably. The app container must remain safely below the 4GB hardware threshold without triggering the Linux OOM-killer mechanism.[30] |
| **Thread Discipline** | Observe CPU core utilization during concurrent requests. | Utilization must scale predictably. The system must not exhibit massive |

| | | |
|---|---|---|
| | | context-switching overhead, confirming the PaddlePaddle thread limiters are successfully suppressing thread explosions.[30] |

Upon the successful and uninterrupted execution of this comprehensive checklist, the local-first, CPU-optimized OCR infrastructure can be certified as fully operational. The pipeline is highly resilient, mathematically optimized for the specific constraints of edge hardware, and fundamentally prepared to serve secure, high-fidelity optical character recognition predictions at scale via labelsontap.ai.

**Works cited**


1. OCR Ranking 2025 – Comparison of the Best Text Recognition and Document Structure Software - Pragmile, accessed April 28, 2026, https://pragmile.com/ocr-ranking-2025-comparison-of-the-best-text-recognition-and-document-structure-software/
2. 7 Best Open-Source OCR Models 2025: Benchmarks & Cost Comparison | E2E Networks, accessed April 28, 2026, https://www.e2enetworks.com/blog/complete-guide-open-source-ocr-models-2025
3. OCR comparison: Tesseract versus EasyOCR vs PaddleOCR vs MMOCR - Toon Beerten, accessed April 28, 2026, https://toon-beerten.medium.com/ocr-comparison-tesseract-versus-easyocr-vs-paddleocr-vs-mmocr-a362d9c79e66
4. cognitiveailab/surya-sci: OCR, layout analysis, reading order, line detection in 90+ languages - GitHub, accessed April 28, 2026, https://github.com/cognitiveailab/surya-sci
5. PaddleOCR vs Tesseract: I Tested Both (2025) | Real Results - CodeSOTA, accessed April 28, 2026, https://www.codesota.com/ocr/paddleocr-vs-tesseract
6. What would you say is currently the most accurate OCR solution if you're not con... | Hacker News, accessed April 28, 2026, https://news.ycombinator.com/item?id=43047121
7. Python Tesseract OCR: Extract text from images using pytesseract - Nutrient, accessed April 28, 2026, https://www.nutrient.io/blog/how-to-use-tesseract-ocr-in-python/
8. Python OCR Tutorial: Tesseract, Pytesseract, and OpenCV - Nanonets, accessed April 28, 2026, https://nanonets.com/blog/ocr-with-tesseract/
9. Does PyTesseract OCR give the confidence of the text detection or text extraction?, accessed April 28, 2026, https://stackoverflow.com/questions/66594679/does-pytesseract-ocr-give-the-confidence-of-the-text-detection-or-text-extractio
10. Optimizing Dockerized FastAPI with TensorFlow: How to reduce a 1.57GB Image Size?, accessed April 28, 2026,

https://www.reddit.com/r/FastAPI/comments/1e1lal6/optimizing_dockerized_fastapi_with_tensorflow_how/
11. Evaluating OCR performance on food packaging labels in South Africa - arXiv, accessed April 28, 2026, https://arxiv.org/html/2510.03570v1
12. EasyOCR Multimodal Model Model: What is, How to Use - Roboflow, accessed April 28, 2026, https://roboflow.com/model/easyocr
13. Technical Analysis of Modern Non-LLM OCR Engines | IntuitionLabs, accessed April 28, 2026, https://intuitionlabs.ai/articles/non-llm-ocr-technologies
14. I accidentally doubled the speed of EasyOCR | by Phuoc Nguyen - Medium, accessed April 28, 2026, https://medium.com/@phuocnguyen90/i-accidentally-doubled-the-speed-of-easyocr-3779ec951424
15. Best OCR Models for Text Recognition in Images - Roboflow Blog, accessed April 28, 2026, https://blog.roboflow.com/best-ocr-models-text-recognition/
16. 8 Top Open-Source OCR Models Compared: A Complete Guide - Modal, accessed April 28, 2026, https://modal.com/blog/8-top-open-source-ocr-models-compared
17. datalab-to/surya: OCR, layout analysis, reading order, table ... - GitHub, accessed April 28, 2026, https://github.com/datalab-to/surya
18. Best ocr : r/LocalLLaMA - Reddit, accessed April 28, 2026, https://www.reddit.com/r/LocalLLaMA/comments/1eecto9/best_ocr/
19. Deep Learning OCR: Tesseract vs docTR Explained with Real-World Results - 47Billion, accessed April 28, 2026, https://47billion.com/blog/deep-learning-ocr-tesseract-vs-doctr-explained-with-real-world-results/
20. Choosing the right model - docTR documentation - GitHub Pages, accessed April 28, 2026, https://mindee.github.io/doctr/latest/using_doctr/using_models.html
21. Tips on Doctr Speed / Performance · mindee doctr · Discussion #1094 - GitHub, accessed April 28, 2026, https://github.com/mindee/doctr/discussions/1094
22. mindee/doctr: docTR (Document Text Recognition) - a seamless, high-performing & accessible library for OCR-related tasks powered by Deep Learning. · GitHub, accessed April 28, 2026, https://github.com/mindee/doctr
23. Exploring docTR — Document OCR with Deep Learning | by Rishika Dwaraghanath, accessed April 28, 2026, https://medium.com/@rthiruve/exploring-doctr-document-ocr-with-deep-learning-539c5a3c6971
24. DeepSeek-OCR-2 vs GLM-OCR vs PaddleOCR: The New Era of Intelligent OCR (2026 Benchmarks) - regolo.ai, accessed April 28, 2026, https://regolo.ai/deepseek-ocr-vs-glm-ocr-vs-paddleocr-benchmark-2026/
25. GitHub - PaddlePaddle/PaddleOCR: Turn any PDF or image document into structured data for your AI. A powerful, lightweight OCR toolkit that bridges the gap between images/PDFs and LLMs. Supports 100+ languages., accessed April 28, 2026, https://github.com/PADDLEPADDLE/PADDLEOCR
26. Paddle OCR Features and Capabilities - LlamaIndex, accessed April 28, 2026, https://www.llamaindex.ai/glossary/what-is-paddleocr

27. 3.3.0 • pypi-paddleocr • tessl • Registry, accessed April 28, 2026, https://tessl.io/registry/tessl/pypi-paddleocr
28. PADDLEOCR. Introduction to OCR | by DhanushKumar - Medium, accessed April 28, 2026, https://medium.com/@danushidk507/paddleocr-439a8d92fb1a
29. What It Really Takes to Use PaddleOCR in Production Systems | by Ankit Ladva | Medium, accessed April 28, 2026, https://medium.com/@ankitladva11/what-it-really-takes-to-use-paddleocr-in-production-systems-d63e38ded55e
30. How to set the flags correctly to reduce CPU usage or GPU usage ? · PaddlePaddle PaddleOCR · Discussion #14497 - GitHub, accessed April 28, 2026, https://github.com/PaddlePaddle/PaddleOCR/discussions/14497
31. General-purpose machine family for Compute Engine - Google Cloud Documentation, accessed April 28, 2026, https://docs.cloud.google.com/compute/docs/general-purpose-machines
32. Droplet Pricing | DigitalOcean, accessed April 28, 2026, https://www.digitalocean.com/pricing/droplets
33. How to Containerize a FastAPI Application with Docker - OneUptime, accessed April 28, 2026, https://oneuptime.com/blog/post/2026-02-08-how-to-containerize-a-fastapi-application-with-docker/view
34. FastAPI in Containers - Docker, accessed April 28, 2026, https://fastapi.tiangolo.com/deployment/docker/
35. PaddleOCR as a Service with FastAPI - YouTube, accessed April 28, 2026, https://www.youtube.com/watch?v=A5JHYMN4-p0
36. Deployments Concepts - FastAPI, accessed April 28, 2026, https://fastapi.tiangolo.com/deployment/concepts/
37. Docker Image Optimization with Slim and Multi-Stage Builds - DZone, accessed April 28, 2026, https://dzone.com/articles/docker-image-optimization-slim-multistage
38. Building Faster, Smaller, and Cleaner Python Docker Images with Multi-Stage Builds, accessed April 28, 2026, https://manabpokhrel7.medium.com/building-faster-smaller-and-cleaner-python-docker-images-with-multi-stage-builds-0da6983a0593
39. Caddy vs Nginx: How Do These Web Servers / Reverse Proxies Compare? - Reddit, accessed April 28, 2026, https://www.reddit.com/r/selfhosted/comments/hur1hx/caddy_vs_nginx_how_do_these_web_servers_reverse/
40. Why Caddy Is My Favorite Reverse Proxy in 2025 | by Hugo Valters | Medium, accessed April 28, 2026, https://blog.valters.eu/why-caddy-is-my-favorite-reverse-proxy-in-2025-9bc41b51b29d
41. How to Use Docker with Caddy for Automatic HTTPS - OneUptime, accessed April 28, 2026, https://oneuptime.com/blog/post/2026-01-16-docker-caddy-automatic-https/view

42. Why Caddy Is My Favorite Reverse Proxy in 2025 - DEV Community, accessed April 28, 2026, https://dev.to/hugovalters/why-caddy-is-my-favorite-reverse-proxy-in-2025-42ed
43. GrantBirki/caddy-fastapi - GitHub, accessed April 28, 2026, https://github.com/GrantBirki/caddy-fastapi
44. Caddy Reverse Proxy in 2025: The Simplest Docker Setup for Your Home Lab, accessed April 28, 2026, https://www.virtualizationhowto.com/2025/09/caddy-reverse-proxy-in-2025-the-simplest-docker-setup-for-your-home-lab/
45. Using a Self-Hosted PDF OCR API with PaddleOCR - DEV Community, accessed April 28, 2026, https://dev.to/edgaras/using-a-self-hosted-pdf-ocr-api-with-paddleocr-3k52
46. How to setup and manage domain and DNS on Cloudflare (Easiest Way)(2026 Guide), accessed April 28, 2026, https://www.youtube.com/watch?v=sx83eEm6nk4
47. CloudFlare AI management with ease - English - n8n Community, accessed April 28, 2026, https://community.n8n.io/t/cloudflare-ai-management-with-ease/162881
48. Onboard a domain · Cloudflare Fundamentals docs, accessed April 28, 2026, https://developers.cloudflare.com/fundamentals/manage-domains/add-site/
49. Manage DNS records - Cloudflare Docs, accessed April 28, 2026, https://developers.cloudflare.com/dns/manage-dns-records/how-to/create-dns-records/
50. How to setup and manage domain and DNS on Cloudflare - YouTube, accessed April 28, 2026, https://www.youtube.com/watch?v=ndzkpFNE6jw
51. Complete Guide to Smoke Testing with Examples - Virtuoso QA, accessed April 28, 2026, https://www.virtuosoqa.com/testing-guides/smoke-testing
52. Smoke Testing 101: Best Practices and Examples - AltexSoft, accessed April 28, 2026, https://www.altexsoft.com/blog/smoke-testing/
53. A Comprehensive Guide to Smoke Testing in Software Development - LaunchDarkly, accessed April 28, 2026, https://launchdarkly.com/blog/comprehensive-guide-smoke-testing-software-development/
54. Integrating Smoke Testing into Your CI/CD Pipeline: What DevOps Needs to Know, accessed April 28, 2026, https://www.harness.io/harness-devops-academy/integrating-smoke-testing-into-your-ci-cd-pipeline-what-devops-needs-to-know
55. How can I make it return every word instead of word groups? · PaddlePaddle PaddleOCR · Discussion #14852 - GitHub, accessed April 28, 2026, https://github.com/PaddlePaddle/PaddleOCR/discussions/14852
56. How To Use OCR Bounding Boxes | by Michael Orozco-Fletcher - Medium, accessed April 28, 2026, https://medium.com/@michael71314/how-to-use-ocr-bounding-boxes-c00303bc11c4